

```
rm(list = ls(all = TRUE))
#Do not delete this!
#It clears all variables to ensure reproducibility
```

PESTICIDE EFFECTIVENESS ON CATERPILLAR EGGS

Caterpillars are considered a pest, as they eat produce on farms. To prevent damage to crops, farmers may need to treat their farms with insecticides, and so this dataset was created to investigate the impacts of different types of insecticide treatments on caterpillar birth rates.

```
eggs <- read.csv("caterpillar_eggs.csv")
head(eggs)
```

```
##   num.eggs farm sprayed lead area
## 1      51   B1      N    N   50
## 2       2   B1      N    N   35
## 3      23   B1      N    N   30
## 4     158   B1      N    N   50
## 5     114   B1      N    N   20
## 6       4   B2      N    N   35
```

A large-scale farming operation carried out an experiment to identify the impacts of two types of insecticides - a) sprayed and b) lead - on the egg laying rate of caterpillars. Different farms were partitioned into small patches of land and treated with i) no insecticide, ii) just sprayed insecticide, iii) just lead insecticide, or iv) both insecticides. After one week, the farmers counted and recorded the number of caterpillar eggs in each patch of land.

The dataset contains:

- farm: the ID of the farm
- area: the area (m²) of the individual patch within the farm
- sprayed: whether or not the patch was sprayed with insecticide (Y for yes; N for no)
- lead: whether or not the patch was treated with lead (Y for yes; N for no)
- num.eggs: the number of caterpillar eggs recorded in the patch of land after one week.

Each row corresponds to a different patch of land. We fit Bayesian count regression models to identify the effect of the insecticides on the laying rate of caterpillars.

Poisson regression model in STAN, using num.eggs as the response variable and area as an offset term.

Before specifying priors for the regression coefficients β , it is useful to explore the distribution of the log rate of caterpillar eggs per unit area, under different treatment combinations.

In our model we will be working with:

$$\log(\mu_i) = \mathbf{x}_i^\top \beta + \log(\text{area}_i)$$

which implies:

$$\log\left(\frac{\mu_i}{\text{area}_i}\right) = \mathbf{x}_i^\top \beta$$

The log of (expected) egg count per m² is equal the linear predictor $\mathbf{x}_i^\top \boldsymbol{\beta}$. Therefore, by computing the log-rates from the observed data, we gain information regarding a reasonable range of values for the $\boldsymbol{\beta}$ coefficients.

To do this, we compute the log-rate of eggs (per unit area) for each treatment category, not considering observations with zero counts to avoid undefined logarithms.

These results help in the selection of weakly informative priors for the $\boldsymbol{\beta}$.

```
#have a look to log (rate eggs per unit of area) to get an idea of priors for beta
```

```
#type of treatment
```

```
no_insecticide <- eggs$sprayed == "N" & eggs$lead == "N"
sprayed_only   <- eggs$sprayed == "Y" & eggs$lead == "N"
lead_only      <- eggs$sprayed == "N" & eggs$lead == "Y"
both_insecticide <- eggs$sprayed == "Y" & eggs$lead == "Y"
```

```
#function to compute log rate (only in case where numb of eggs >0)
```

```
log_rate_summary <- function(filter) {
  idx <- which(filter & eggs$num.eggs > 0)
  log_rate <- log(eggs$num.eggs[idx] / eggs$area[idx])
  summary(log_rate)
}
```

```
#summary for each case:
```

```
cat("No insecticide:\n")
```

```
## No insecticide:
```

```
print(log_rate_summary(no_insecticide))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -3.91202 -0.57873  0.15415 -0.05823  0.96188  2.17475
```

```
cat("\nSprayed only:\n")
```

```
##
```

```
## Sprayed only:
```

```
print(log_rate_summary(sprayed_only))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -3.9120 -1.8971 -0.2231 -0.7798  0.1570  1.3813
```

```
cat("\nLead only:\n")
```

```
##
```

```
## Lead only:
```

```
print(log_rate_summary(lead_only))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -3.55535 -1.37649 -0.02899 -0.39308  0.69315  1.62137
```

```
cat("\nBoth insecticides:\n")
```

```
##
## Both insecticides:
```

```
print(log_rate_summary(both_insecticide))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -3.91202 -1.94591 -0.91629 -0.96647  0.05557  1.32176
```

After excluding zero observations, we observed that most log-rates ranged between -4 and +2, with medians varying by treatment group, from around +0.15 with no insecticide to about -0.9 when both insecticides were applied. Indicating that insecticides seem to lower the rate of eggs.

Given this range, we chose a **Normal(0, 5²)** prior for the regression coefficients.

This is a weakly informative prior that allows effect of the insecticides while still centering around no effect. It allows for large treatment effects, both increases and decreases in the log-rate, while preventing values to be too high.

```
library(rstan)
```

```
## Warning: package 'rstan' was built under R version 4.4.3
```

```
## Loading required package: StanHeaders
```

```
##
## rstan version 2.32.7 (Stan version 2.32.2)
```

```
## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using 'reduce_sum()' or 'map_rect()' Stan functions,
## change 'threads_per_chain' option:
## rstan_options(threads_per_chain = 1)
```

```
## Do not specify '-march=native' in 'LOCAL_CPPFLAGS' or a Makevars file
```

```
library(coda)
```

```
##
## Attaching package: 'coda'

## The following object is masked from 'package:rstan':
##
##      traceplot
```

```

q1_mod1_string <-"
data {
  int<lower=1> N;           // number of observations
  int<lower=1> K;           // columns of X (design matrix)
  matrix[N, K] X;          // model matrix
  vector[N] log_area;      // offset
  int<lower=0> num_eggs[N]; // response
}

parameters {
  vector[K] beta;          // coeff
}

model {
  vector[N] eta; //linear pred

  // priors for beta
  for(i in 1:K){
    beta[i] ~ normal(0, 5); //Normal uses sigma in stan
  }

  eta = X * beta + log_area; // matrix vector product + offset

  //likelihood

  for(i in 1:N) {
    num_eggs[i] ~ poisson(exp(eta[i]));
  }
}

generated quantities {
  vector[N] eta;
  vector[N] mu;
  vector[N] log_like;

  eta = X * beta + log_area;

  mu = exp(eta);

  for (i in 1:N){
    log_like[i] = poisson_lpmf(num_eggs[i] | mu[i]); //log-like for 1.3)
  }
}

"

#make sure factors in the data:
eggs$farm=as.factor(eggs$farm)
eggs$sprayed=as.factor(eggs$sprayed)
eggs$lead=as.factor(eggs$lead)

#build model matrix (including intercept)

```

```
X <- model.matrix(~ sprayed + lead + sprayed * lead, data = eggs)
```

```
#function for initial data
```

```
init_fun <- function() {  
  list(beta = rnorm(ncol(X), mean = 0, sd = 5)) #use the prior  
}
```

```
data_q1_m1 <- list(  
  N = nrow(eggs),  
  K = ncol(X),  
  num_eggs = eggs$num.eggs,  
  X = X,  
  log_area = log(eggs$area))
```

```
fname_q1_m1 <- 'q1_m1.stan'  
cat(q1_mod1_string, file = fname_q1_m1, append = FALSE)
```

```
#fit
```

```
res_q1_m1 <- stan(  
  file = fname_q1_m1,  
  data = data_q1_m1,  
  chains = 3,  
  iter = 3000,  
  warmup = 1000,  
  cores = parallel::detectCores(),  
  init= init_fun)
```

```
#turn into mcmc object for diagnostics
```

```
mcmc_q1_m1 <- As.mcmc.list(res_q1_m1, pars = "beta")
```

```
#effective size
```

```
effectiveSize(mcmc_q1_m1)
```

```
## beta[1] beta[2] beta[3] beta[4]  
## 2573.798 2567.964 2477.438 2321.706
```

```
#Gelman-Rubin
```

```
gelman.diag(mcmc_q1_m1)
```

```
## Potential scale reduction factors:
```

```
##
```

```
## Point est. Upper C.I.
```

```
## beta[1] 1 1
```

```
## beta[2] 1 1
```

```
## beta[3] 1 1
```

```
## beta[4] 1 1
```

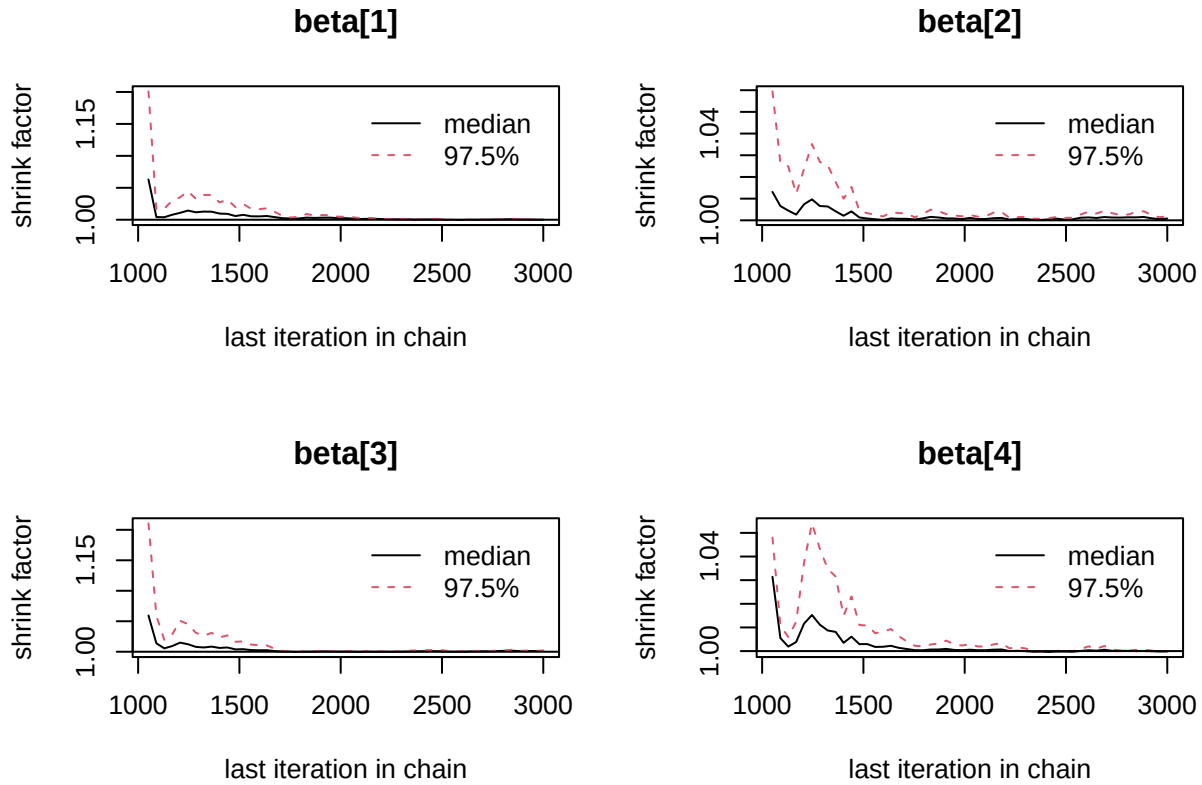
```
##
```

```
## Multivariate psrf
```

```
##
```

```
## 1
```

```
gelman.plot(mcmc_q1_m1)
```



To evaluate the impact of insecticide treatments on the number of caterpillar eggs, we fit a Bayesian Poisson regression model using STAN.

The response variable is the number of caterpillar eggs observed on a patch of land after one week. We include fixed effects for the use of sprayed and lead insecticides, along with their interaction, and use the area of each field as log-offset term.

Let:

- y_i be the number of eggs in patch i
- area_i be the area (in m^2) of patch i
- $\mathbf{x}_i \in \mathbb{R}^K$ be the covariate vector for patch i
- $\boldsymbol{\beta} \in \mathbb{R}^K$ be the vector of regression coefficients

The model is:

$$y_i \sim \text{Poisson}(\mu_i), \quad \text{with} \quad \log(\mu_i) = \mathbf{x}_i^\top \boldsymbol{\beta} + \log(\text{area}_i)$$

or equivalently:

$$\mu_i = \exp(\mathbf{x}_i^\top \boldsymbol{\beta}) \cdot \text{area}_i$$

The expected number of eggs is then proportional to the area, and $\exp(\mathbf{x}_i^\top \boldsymbol{\beta})$ is the expected number of eggs per unit of area.

We use weakly informative priors for the regression coefficients:

$$\beta_k \sim \mathcal{N}(0, 5^2)$$

This prior centers each coefficient around zero (no effect), while allowing flexibility for both positive and negative effects.

We ran the model using: 3 chains, 3000 iterations per chain (1000 warmup) and initial values for each β_k drawn randomly from the prior: $\mathcal{N}(0, 5^2)$

The posterior output was analyzed using the coda package. The following results were obtained for the four regression coefficients:

-All parameters have effective sizes above 2000

-All values of the Gelman-Rubin statistics are close to 1, not indicating problems of not convergence across chains.

-The Gelman-Rubin plots show that the statistics are already quite close to 1 after warm-up, after performing the final iterations of the chains all factors converge to 1..

```
colnames(X) #useful to interpret summary
```

```
## [1] "(Intercept)"      "sprayedY"          "leadY"             "sprayedY:leadY"
```

```
#summary for parameters
```

```
print(res_q1_m1, pars = "beta")
```

```
## Inference for Stan model: anon_model.
```

```
## 3 chains, each with iter=3000; warmup=1000; thin=1;
```

```
## post-warmup draws per chain=2000, total post-warmup draws=6000.
```

```
##
```

##		mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
##	beta[1]	0.38	0	0.01	0.36	0.37	0.38	0.39	0.40	2396	1
##	beta[2]	-0.96	0	0.01	-0.98	-0.97	-0.96	-0.95	-0.93	2425	1
##	beta[3]	-0.46	0	0.01	-0.48	-0.46	-0.46	-0.45	-0.43	2347	1
##	beta[4]	0.34	0	0.02	0.30	0.33	0.34	0.36	0.38	2175	1

```
##
```

```
## Samples were drawn using NUTS(diag_e) at Fri May 23 12:02:10 2025.
```

```
## For each parameter, n_eff is a crude measure of effective sample size,
```

```
## and Rhat is the potential scale reduction factor on split chains (at
```

```
## convergence, Rhat=1).
```

```
mean_posterior_beta <- summary(res_q1_m1, pars = "beta")$summary[, "mean"]
```

```
#expected eggs without insecticides per unit of area
```

```
exp(mean_posterior_beta[1])
```

```
## beta[1]
```

```
## 1.462612
```

```
#factor multiplying baseline number of eggs (per unit of area) when sprayed YES (lead NO)
exp(mean_posterior_beta[2])
```

```
## beta[2]
## 0.384594
```

```
#factor multiplying baseline number of eggs (per unit of area) when lead YES (sprayed NO)
exp(mean_posterior_beta[3])
```

```
## beta[3]
## 0.6342835
```

```
#factor multiplying baseline number of eggs (per unit of area) when both yes
exp(sum(mean_posterior_beta[2:4]))
```

```
## [1] 0.3427997
```

The model includes an intercept and fixed effects for sprayed, lead, and their interaction.

- beta[1] (Intercept) -> posterior mean: 0.38.
This represents the expected log egg rate per m² when no insecticide is used.
On the original scale:

$$\exp(0.38) \approx 1.46 \text{ eggs per m}^2 \text{ (baseline)}$$

Without insecticides we expected 1.46 eggs per squared meter.

Since the model uses a log link function, the exponential of the coefficients for the insecticides correspond to the multiplicative factor to the expected number of eggs without any treatment.

- beta[2] (Sprayed only) -> posterior mean: -0.96
Applying only the sprayed insecticide reduces the expected rate of:

$$\exp(-0.96) \approx 0.38$$

Then we expect $0.38 \times 1.46 = 0.55$ eggs per squared meter, in the case we only use the sprayed insecticide.

- beta[3] (Lead only) -> posterior mean: -0.46
Applying only lead reduces the expected rate of:

$$\exp(-0.46) \approx 0.63$$

Then we expect $0.63 \times 1.46 = 0.92$ eggs per squared meter, in the case we only use the lead insecticide.

- beta[4] (Interaction) -> posterior mean: +0.34
When both insecticides are applied together, the total expected log effect is:

$$-0.96 - 0.46 + 0.34 = -1.08$$

and then:

$$\exp(-1.08) \approx 0.34$$

Then we expect $0.34 \times 1.46 = 0.50$ eggs per squared meter, in the case we use both the lead and sprayed insecticide.

The results show that both insecticides significantly reduce the expected number of eggs, with sprayed having a stronger individual effect than lead (greater mean for sprayed and also credible intervals do not overlap).

When both insecticides are used, the egg rate is reduced even more, although the positive sign of the interaction coefficient suggests the combined effect is less than the sum of the individual effects. However, the combined treatment gives back the lowest expected egg rate overall.

```
#posterior densities of the coefficients beta  
library(bayesplot)
```

```
## This is bayesplot version 1.11.1
```

```
## - Online documentation and vignettes at mc-stan.org/bayesplot
```

```
## - bayesplot theme set to bayesplot::theme_default()
```

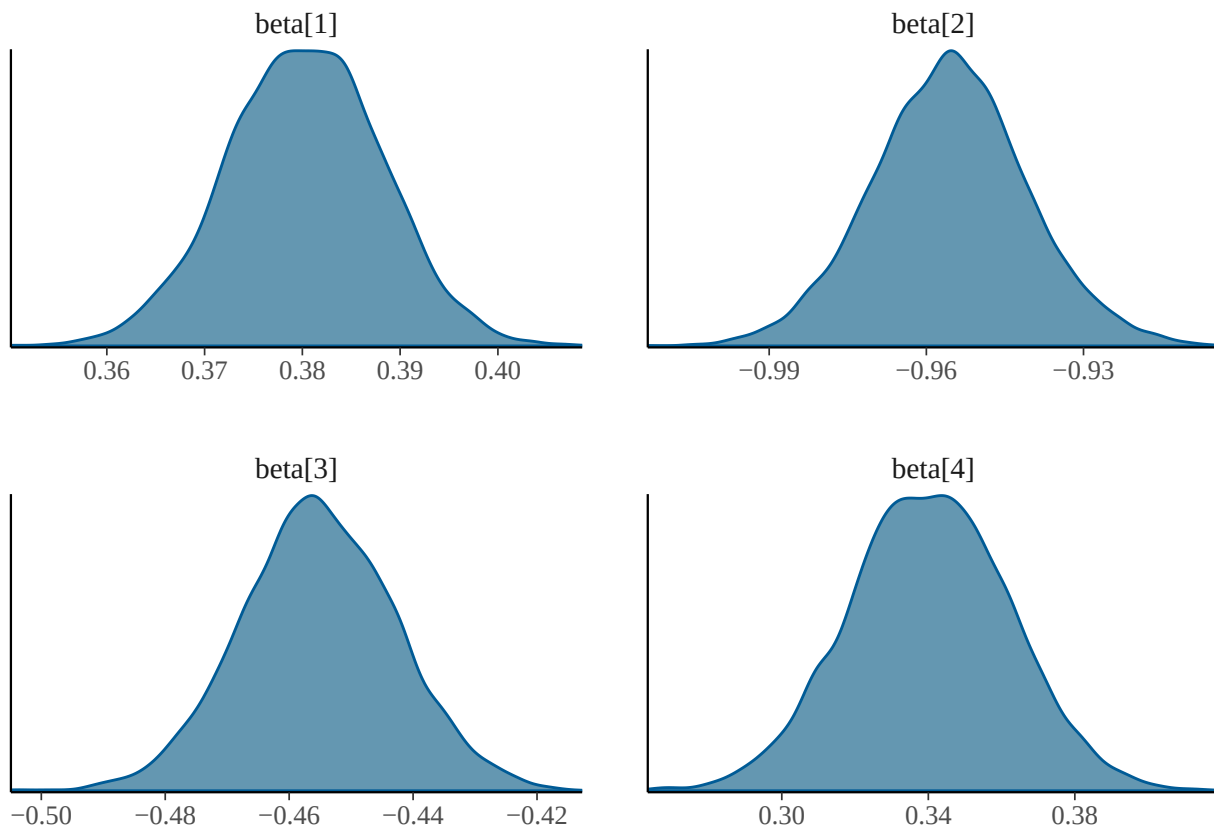
```
## * Does _not_ affect other ggplot2 plots
```

```
## * See ?bayesplot_theme_set for details on theme setting
```

```
names(res_q1_m1)[1:4]
```

```
## [1] "beta[1]" "beta[2]" "beta[3]" "beta[4]"
```

```
mcmc_dens(res_q1_m1, pars = c("beta[1]", "beta[2]", "beta[3]", "beta[4]"))
```



Posterior densities show quite narrow (compared to the prior) and different domains for each β_i , indicating a different and quite clear effect in each case, driven by the data.

Now we compare the use of both insecticides with only the sprayed one:

```
dim(extract(res_q1_m1)$beta)

## [1] 6000    4

colnames(X)

## [1] "(Intercept)"    "sprayedY"        "leadY"          "sprayedY:leadY"

#extract beta samples
beta_samples <- extract(res_q1_m1)$beta

#1st col: intercept, 2nd: sprayedY, 3rd : leadY, 4th: sprayedY:leadY
ind_int <- 1
ind_sprayed <- 2
ind_lead <- 3
ind_both <- 4

# expected number of eggs (per unit area) for both
rate_both <- exp(beta_samples[, ind_int] + beta_samples[, ind_sprayed]
                 + beta_samples[, ind_lead] + beta_samples[, ind_both])

# expected number of eggs (per unit area) for spray
rate_spray <- exp(beta_samples[, ind_int] + beta_samples[, ind_sprayed])

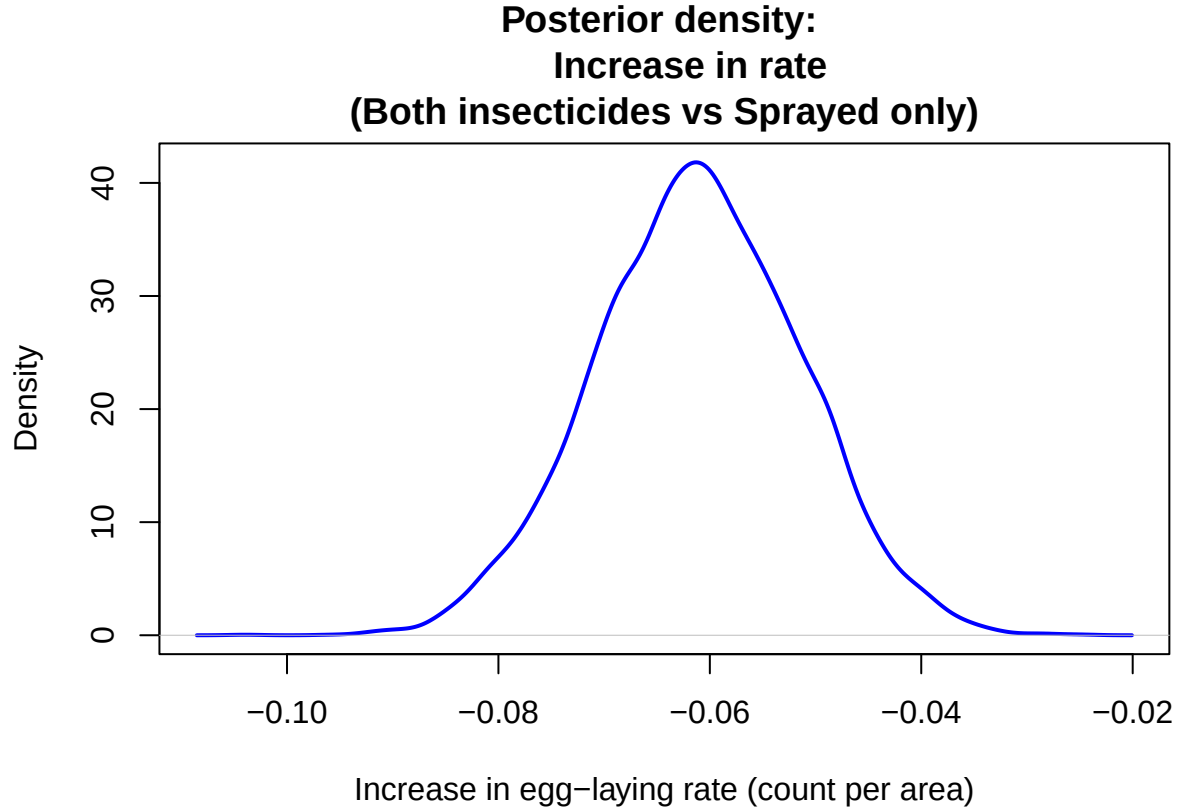
#difference in sprayed
rate_diff <- rate_both - rate_spray

mean_rate_diff <- mean(rate_diff)

cat("Mean of the posterior samples of the difference: \n",
    mean_rate_diff)

## Mean of the posterior samples of the difference:
## -0.0611274

plot(density(rate_diff),
     main = "Posterior density:
Increase in rate\n(Both insecticides vs Sprayed only)",
     xlab = "Increase in egg-laying rate (count per area)",
     col = "blue",
     lwd = 2)
```



We compared the expected egg rate per unit area when using both insecticides versus using only the sprayed one. The goal is to understand if combining the two treatments allows a greater reduction in egg rate than using only the sprayed or not.

The expected egg rate per unit of area in the Poisson model is given by:

$$\mu = \exp(\mathbf{x}_i^\top \boldsymbol{\beta})$$

For each posterior sample, we computed:

Rate under sprayed only:

$$\mu_{\text{sprayed}} = \exp(\beta_{\text{intercept}} + \beta_{\text{spray}})$$

Rate under both insecticides:

$$\mu_{\text{both}} = \exp(\beta_{\text{intercept}} + \beta_{\text{spray}} + \beta_{\text{lead}} + \beta_{\text{interaction}})$$

Difference:

$$\Delta = \mu_{\text{both}} - \mu_{\text{sprayed}}$$

then plotted the posterior density of Δ across all posterior draws.

The posterior density of Δ lies entirely in negative values, with values approximately from -0.10 to -0.02. Indicating that the use of both insecticides is supported statistically as being more effective than using only the sprayed one (negative values for Δ mean that the expected rate of eggs per squared meter is lower when using both insecticides).

The mode of the distribution is around -0.06, meaning that in the case of using both insecticides we usually expect 0.06 less eggs per squared meter, compared to using only the sprayed one. This might not be a big difference over one squared meter, but over large fields could be an important effect.

Even though the posterior credible interval of the coefficient $\beta_{interaction}$ is all positive, the overall effect of combining both treatments still leads to a greater reduction in the egg rate than using sprayed alone.

Combined treatment is more effective.

To account for overdispersion (<https://en.wikipedia.org/wiki/Overdispersion>), we use a negative-binomial regression model. Using “skeptical” prior for the negative-binomial hyperparameter.

```
#https://mc-stan.org/docs/2_21/functions-reference/negative-binomial-distribution.html
```

```
#https://mc-stan.org/docs/2_21/functions-reference/nbalt.html#nbalt
```

```
#to get a sense of the order of phi we need to
```

```
#use to cancel the effect of overdispersion (skeptical prior)
```

```
mean(eggs$num.eggs)
```

```
## [1] 29.03077
```

```
q1_mod2_string <-"
```

```
data {
```

```
  int<lower=1> N;
```

```
  int<lower=1> K;
```

```
  matrix[N, K] X;
```

```
  vector[N] log_area;
```

```
  int<lower=0> num_eggs[N];
```

```
}
```

```
parameters {
```

```
  vector[K] beta;
```

```
  real<lower=0> phi; // overdispersion parameter
```

```
}
```

```
model {
```

```
  vector[N] eta;
```

```
  vector[N] mu;
```

```
  // Priors
```

```
  beta ~ normal(0, 5); //same priors as before
```

```
  phi ~ gamma(2, 0.001); // skeptical: favors large phi (small overdispersion)
```

```
  // Linear predictor
```

```
  eta = X * beta + log_area;
```

```
  mu = exp(eta);
```

```
  for(i in 1:N){
```

```
    num_eggs[i] ~ neg_binomial_2(mu[i], phi);
```

```
  }
```

```
}
```

```
generated quantities {
```

```
  vector[N] mu_rep;
```

```
  vector[N] log_like;
```

```
  mu_rep = exp(X * beta + log_area);
```

```
  for (i in 1:N){
```

```
    log_like[i] = neg_binomial_2_lpmf(num_eggs[i] | mu_rep[i], phi); //log-like
```

```

}
}
"

data_q1_m2 <- list(
  N = nrow(eggs),
  K = ncol(X),
  num_eggs = eggs$num.eggs,
  X = X,
  log_area = log(eggs$area))

fname_q1_m2 <- 'q1_m2.stan'
cat(q1_mod2_string, file = fname_q1_m2, append = FALSE)

init_fun <- function() { #for initial values
  list(beta = rnorm(ncol(X), mean = 0, sd = 5),
        phi = rgamma(1, shape = 2, rate = 0.001))
}

#fit
res_q1_m2 <- stan(
  file = fname_q1_m2,
  data = data_q1_m2,
  chains = 3,
  iter = 3000,
  warmup = 1000,
  cores = parallel::detectCores(),
  init = init_fun)

```

To account for potential overdispersion in the caterpillar egg data, we extended the Poisson regression model by fitting a Negative Binomial regression model.

While the Poisson model assumes that the mean and variance of the response are equal, count data often exhibit variability that increases with the mean, which Poisson model doesn't allow. This is known as overdispersion.

The Poisson distribution has only one parameter (μ) and:

$$\mathbb{E}[y] = \mu, \quad \text{Var}[y] = \mu$$

We use a **Negative Binomial model**, which introduces an overdispersion parameter ϕ to allow the variance to exceed the mean:

$$\mathbb{E}[y] = \mu, \quad \text{Var}[y] = \mu + \frac{\mu^2}{\phi}$$

When $\phi \rightarrow \infty$, the variance converges to μ , and the Negative Binomial reduces to the Poisson model. Smaller values of ϕ correspond to greater overdispersion.

Let:

- y_i be the number of eggs on patch i
- $\mu_i = \exp(\mathbf{x}_i^\top \boldsymbol{\beta}) \cdot \text{area}_i$ be the expected number of eggs on the patch

- ϕ be the overdispersion parameter

Then the new model is:

$$y_i \sim \text{NegBinomial}(\mu_i, \phi)$$

$$\log(\mu_i) = \mathbf{x}_i^\top \boldsymbol{\beta} + \log(\text{area}_i)$$

We use the `neg_binomial_2()` function in Stan, which directly parameterise the negative binomial using the mean μ and the overdispersion parameter ϕ .

The fixed effect coefficients $\boldsymbol{\beta}$ are assigned weakly informative Normal priors, the same as before:

$$\beta_k \sim \mathcal{N}(0, 5^2)$$

The overdispersion parameter ϕ is assigned a skeptical prior $\text{Gamma}(2, 0.001)$ prior, which has a mean of 2000, a 5th percentile around 39, and a 95th percentile above 5200. An alternative choice would have been setting a exponential prior on $1/\phi$ (which has a 0 mode, giving us a skeptical prior).

Given that the observed mean number of eggs is approximately 29, then the mean squared is around $29^2 = 841$. In order to be skeptical, we would need $\phi \gg 841$, since the Negative Binomial variance is

$$\text{Var}(y) = \mu + \frac{\mu^2}{\phi},$$

The $\text{Gamma}(2, 0.001)$ prior, places a big probability mass on values of ϕ that imply small overdispersion, making it a skeptical prior that favors the simpler Poisson model unless the data strongly suggest otherwise.

We used 3 chains, each with 3000 iterations (1000 warmup).

- $\boldsymbol{\beta}$ coefficients were initialized from $\mathcal{N}(0, 5^2)$

- ϕ was initialized from $\text{Gamma}(2, 0.001)$

All initial values are consistent with the specified priors.

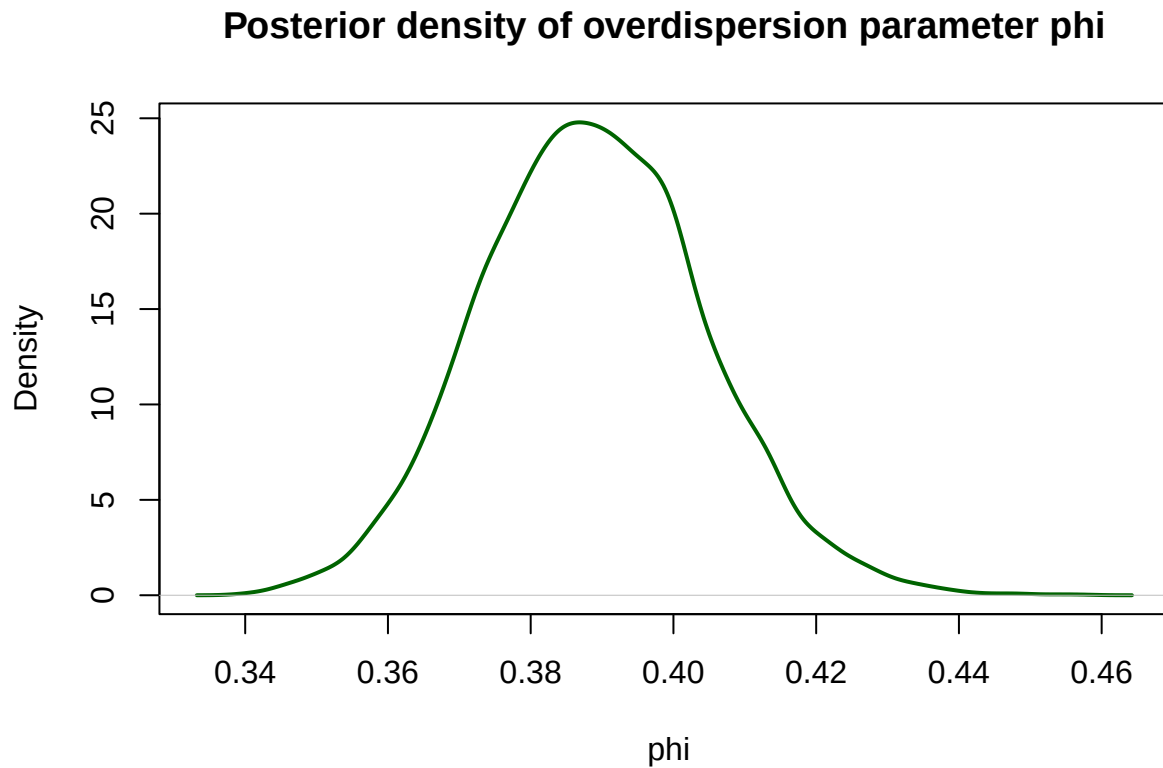
```
print(res_q1_m2, par='phi')
```

```
## Inference for Stan model: anon_model.
## 3 chains, each with iter=3000; warmup=1000; thin=1;
## post-warmup draws per chain=2000, total post-warmup draws=6000.
##
##      mean se_mean   sd 2.5% 25% 50% 75% 97.5% n_eff Rhat
## phi 0.39         0 0.02 0.36 0.38 0.39 0.4  0.42 4002    1
##
## Samples were drawn using NUTS(diag_e) at Fri May 23 12:07:20 2025.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
post_nb <- extract(res_q1_m2)
phi_samples <- post_nb$phi

plot(density(phi_samples),
```

```
main = "Posterior density of overdispersion parameter phi",
xlab = " phi",
col = "darkgreen", lwd = 2)
```



#small phi even with really skeptical prior, indicating over dispersion is quite high

We examined the posterior distribution of the overdispersion parameter ϕ from the new model. As discussed earlier: a large value of ϕ corresponds to a variance close to the Poisson model ($\text{Var}(y_i) \approx \mu_i$), while a small ϕ indicates strong overdispersion.

Despite placing a really skeptical prior on $\phi \sim \text{Gamma}(2, 0.001)$, the posterior distribution is concentrated around:

$$\phi \approx 0.39$$

and the posterior distribution lies entirely in a low range of values (approximately between 0.32 and 0.45).

This implies that:

- The variance is much larger than the mean across observations ($\text{Var}(y_i) \gg \mu_i$)
- There is strong evidence of overdispersion in the data and the prior belief was clearly changed by the data
- the Poisson model is strongly underestimating variability in the dataset

The result also validate the choice of $\text{Gamma}(2, 0.001)$ as an appropriate prior in this case.

Compare models using WAIC

```
#FIRST MODEL (poisson)
data_q1_m1_like <- list(
  N = nrow(eggs),
  K = ncol(X),
  num_eggs = eggs$num.eggs,
  X = X,
  log_area = log(eggs$area))

fit_m1_like <- stan(
  file = fname_q1_m1,
  data = data_q1_m1_like,
  chains = 1,
  iter = 11000, #to get 2500 samples with thin = 4
  thin = 4,
  warmup=1000,
  cores = parallel::detectCores())

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.00105 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 10.5 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 11000 [ 0%] (Warmup)
## Chain 1: Iteration: 1001 / 11000 [ 9%] (Sampling)
## Chain 1: Iteration: 2100 / 11000 [19%] (Sampling)
## Chain 1: Iteration: 3200 / 11000 [29%] (Sampling)
## Chain 1: Iteration: 4300 / 11000 [39%] (Sampling)
## Chain 1: Iteration: 5400 / 11000 [49%] (Sampling)
## Chain 1: Iteration: 6500 / 11000 [59%] (Sampling)
## Chain 1: Iteration: 7600 / 11000 [69%] (Sampling)
## Chain 1: Iteration: 8700 / 11000 [79%] (Sampling)
## Chain 1: Iteration: 9800 / 11000 [89%] (Sampling)
## Chain 1: Iteration: 10900 / 11000 [99%] (Sampling)
## Chain 1: Iteration: 11000 / 11000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 6.508 seconds (Warm-up)
## Chain 1:                70.407 seconds (Sampling)
## Chain 1:                76.915 seconds (Total)
## Chain 1:
```

#SECOND MODEL (inverse binomial)

```
data_q1_m2_like <- list(
  N = nrow(eggs),
  K = ncol(X),
  num_eggs = eggs$num.eggs,
  X = X,
```

```

log_area = log(eggs$area))

fit_m2_like <- stan(
  file = fname_q1_m2,
  data = data_q1_m2_like,
  chains = 1,
  iter = 11000, #to get 2500 samples with thin = 4
  thin = 4,
  warmup=1000,
  cores = parallel::detectCores()
)

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.00078 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 7.8 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 11000 [ 0%] (Warmup)
## Chain 1: Iteration: 1001 / 11000 [ 9%] (Sampling)
## Chain 1: Iteration: 2100 / 11000 [19%] (Sampling)
## Chain 1: Iteration: 3200 / 11000 [29%] (Sampling)
## Chain 1: Iteration: 4300 / 11000 [39%] (Sampling)
## Chain 1: Iteration: 5400 / 11000 [49%] (Sampling)
## Chain 1: Iteration: 6500 / 11000 [59%] (Sampling)
## Chain 1: Iteration: 7600 / 11000 [69%] (Sampling)
## Chain 1: Iteration: 8700 / 11000 [79%] (Sampling)
## Chain 1: Iteration: 9800 / 11000 [89%] (Sampling)
## Chain 1: Iteration: 10900 / 11000 [99%] (Sampling)
## Chain 1: Iteration: 11000 / 11000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 10.268 seconds (Warm-up)
## Chain 1:           97.08 seconds (Sampling)
## Chain 1:           107.348 seconds (Total)
## Chain 1:

library(loo)

## This is loo version 2.8.0

## - Online documentation and vignettes at mc-stan.org/loo

## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use the 'cores' ar

## - Windows 10 users: loo may be very slow if 'mc.cores' is set in your .Rprofile file (see https://gi

##
## Attaching package: 'loo'

```

```
## The following object is masked from 'package:rstan':
##
##      loo
```

```
#help(waic) -> waic requires matrix of likelihood (S x N), where S = num of samples, N = num of data po
```

```
#get log likelihood from stan models
log_like_poisson <- extract(fit_m1_like)$log_like
log_like_nb      <- extract(fit_m2_like)$log_like

dim(log_like_poisson) #ok 2500 samples, 1300 data points (ok looking at help(waic))
```

```
## [1] 2500 1300
```

```
dim(log_like_nb) #ok
```

```
## [1] 2500 1300
```

```
#WAIC
waic_poisson <- waic(log_like_poisson)
```

```
## Warning:
## 87 (6.7%) p_waic estimates greater than 0.4. We recommend trying loo instead.
```

```
waic_nb      <- waic(log_like_nb)

cat('\n Poisson WAIC: \n')
```

```
##
## Poisson WAIC:
```

```
print(waic_poisson$estimates['waic', 'Estimate'])
```

```
## [1] 55041.52
```

```
cat("\n Negative Binomial WAIC:\n")
```

```
##
## Negative Binomial WAIC:
```

```
print(waic_nb$estimates['waic', 'Estimate'])
```

```
## [1] 10408.13
```

To compare the Poisson and the Negative Binomial models, we computed the WAIC for each using the loo package. The WAIC is a criterion that estimates predictive accuracy, based on the posterior log-likelihood. For both models, we generated 2500 posterior samples using 1 chain with iter = 11000, thin = 4, and warmup = 1000.

In both cases, the log-likelihoods for each observation in each sample have been stored in a `log_like` variable in the generated quantities block.

The resulting `log_like` matrices, extracted from the posterior samples, have shape 2500×1300 (samples $\times$ observations).

We computed the WAIC using the `waic()` function:

- **Poisson model WAIC:** around 55000
- **Negative Binomial model WAIC:** around 10500

The WAIC for the Negative Binomial model is much lower than the one from the Poisson model. Since lower is better with WAIC, this clearly shows that the Negative Binomial model is a better fit to the data.

This is consistent with what we observed earlier: the data exhibit strong overdispersion, which the Poisson model doesn't keep track of. The Negative Binomial model, by introducing an overdispersion parameter, is a better choice in our case.

Mixed effects count regression model to include a farm-specific random effect.

```
q1_mod3_string <-"
data {
  int<lower=1> N;                // observations
  int<lower=1> K;                // number fixed coeff
  matrix[N, K] X;              // design matrix
  vector[N] log_area;          // offset
  int<lower=0> num_eggs[N];      // response
  int<lower=1> J;               // number of farms
  int<lower=1, upper=J> farm_id[N]; // farm for each obs
}

parameters {
  vector[K] beta;              // fixed effects
  real<lower=0> phi;            // overdispersion
  vector[J] theta_farm;        // random effects per farm
  real mu_theta;               // mean random effects
  real<lower=0> sigma_theta;    // sd random effect
}

model {
  vector[N] eta;               //linear predictor
  vector[N] mu;                //mean per observation

  // Priors

  for(i in 1:K){
    beta[i] ~ normal(0, 5);
  }

  phi ~ gamma(2, 0.001);

  //random effects
  mu_theta ~ normal(0, 10);
  sigma_theta ~ gamma(2, 0.1);

  for( j in 1:J){
    theta_farm[j] ~ normal(mu_theta, sigma_theta);
  }

  for(i in 1:N){
    eta[i] = X[i,] * beta + log_area[i] + theta_farm[ farm_id[i] ] ;
    mu[i] = exp(eta[i]);
    num_eggs[i] ~ neg_binomial_2( mu[i], phi);
  }
}

generated quantities {
  vector[N] mu_rep;
  int<lower=0> y_rep[N]; // posterior predictive sample

  for (i in 1:N) {
```

```

    mu_rep[i] = exp( (X[i,] * beta) + log_area[i] + theta_farm[farm_id[i]]);
    y_rep[i] = neg_binomial_2_rng(mu_rep[i], phi);
  }
}

"

#design matrix no intercept
X0 <- X[,2:4]

#number of betas in new matrix
K0 <- ncol(X0)

#farm as a number (making sure B1 -> 1, B2 ->2,...)
eggs$farm <- factor(eggs$farm, levels = paste0("B", 1:13))
eggs$farm_id <- as.integer(eggs$farm)
J <- length(unique(eggs$farm_id)) #number of different farms in the dataset

#data new model
data_q1_m3 <- list(
  N = nrow(eggs),
  K = K0,
  X = X0,
  log_area = log(eggs$area),
  num_eggs = eggs$num.eggs,
  J = J,
  farm_id = eggs$farm_id)

fname_q1_m3 <- 'q1_m3.stan'
cat(q1_mod3_string, file = fname_q1_m3, append = FALSE)

#fit
res_q1_m3 <- stan(
  file = fname_q1_m3,
  data = data_q1_m3,
  chains = 1,
  iter = 6000,
  pars = c("beta", "phi", "theta_farm", "y_rep"))

```

```

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.000759 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 7.59 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 6000 [  0%] (Warmup)
## Chain 1: Iteration:   600 / 6000 [ 10%] (Warmup)
## Chain 1: Iteration:  1200 / 6000 [ 20%] (Warmup)
## Chain 1: Iteration:  1800 / 6000 [ 30%] (Warmup)
## Chain 1: Iteration:  2400 / 6000 [ 40%] (Warmup)

```

```

## Chain 1: Iteration: 3000 / 6000 [ 50%] (Warmup)
## Chain 1: Iteration: 3001 / 6000 [ 50%] (Sampling)
## Chain 1: Iteration: 3600 / 6000 [ 60%] (Sampling)
## Chain 1: Iteration: 4200 / 6000 [ 70%] (Sampling)
## Chain 1: Iteration: 4800 / 6000 [ 80%] (Sampling)
## Chain 1: Iteration: 5400 / 6000 [ 90%] (Sampling)
## Chain 1: Iteration: 6000 / 6000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 21.39 seconds (Warm-up)
## Chain 1: 26.658 seconds (Sampling)
## Chain 1: 48.048 seconds (Total)
## Chain 1:

```

Explanation:

Let y_i be the number of caterpillar eggs observed in patch i , and let area_i be the area (in m^2) of that patch. The model assumes:

$$y_i \sim \text{NegBinomial}(\mu_i, \phi)$$

$$\mu_i = \exp(\mathbf{x}_i^\top \boldsymbol{\beta} + \log(\text{area}_i) + \theta_{j[i]})$$

where:

- $\mathbf{x}_i$ is the row of the design matrix for fixed effects (sprayed, lead, and their interaction. No intercept anymore).
- $\boldsymbol{\beta} = (\beta_{\text{spray}}, \beta_{\text{lead}}, \beta_{\text{interaction}})$ are the fixed effect coefficients (no intercept)
- $\theta_{j[i]}$ is the random effect for farm $j[i]$ that is the farm of observation i . To simplify, now each farm as its own intercept.

The random effects are modeled hierarchically:

$$\theta_j \sim \mathcal{N}(\mu_\theta, \sigma_\theta^2) \quad \text{for } j = 1, \dots, 13$$

with the following priors:

$$\beta_k \sim \mathcal{N}(0, 5^2) \quad \text{for } k = 1, 2, 3$$

$$\phi \sim \text{Gamma}(2, 0.001)$$

$$\mu_\theta \sim \mathcal{N}(0, 10^2)$$

$$\sigma_\theta \sim \text{Gamma}(2, 0.1)$$

This new model keeps track of the between-farm variability through random effects.

The response variable is modeled, again, using a Negative Binomial distribution with mean μ_i and overdispersion parameter ϕ .

The prior for ϕ and $\boldsymbol{\beta}$ are the same used before.

The role of θ_j in this model is similar to that of an intercept in the initial model, as it captures the baseline number of eggs (now different in each farm) when not using insecticides.

Each farm-specific random effect θ_j is assigned a Normal distribution, as done before for the intercept. However, in this case, the distribution is centered around mean μ_θ , which itself is given a prior:

$$\mu_\theta \sim \mathcal{N}(0, 10^2).$$

This reflects limited prior knowledge (we know the intercept before was close to 0), but this prior allows μ_θ to vary substantially across the log scale.

The standard deviation σ_θ , which governs the amount of variability between farms, is also treated hierarchically and assigned a

$$\sigma_\theta \sim \text{Gamma}(2, 0.1)$$

prior.

This prior has mean of 20 and places 95% of its mass roughly between 0.36 and 60. This reflects a belief that the variability between farms could be small, moderate, or even quite large, allowing the data to determine the difference between each farm.

Now let's have a look to the posterior predictive checks:

```
library(bayesplot)
library(fBasics)
library(ggplot2)

#response
y_obs <- eggs$num.eggs

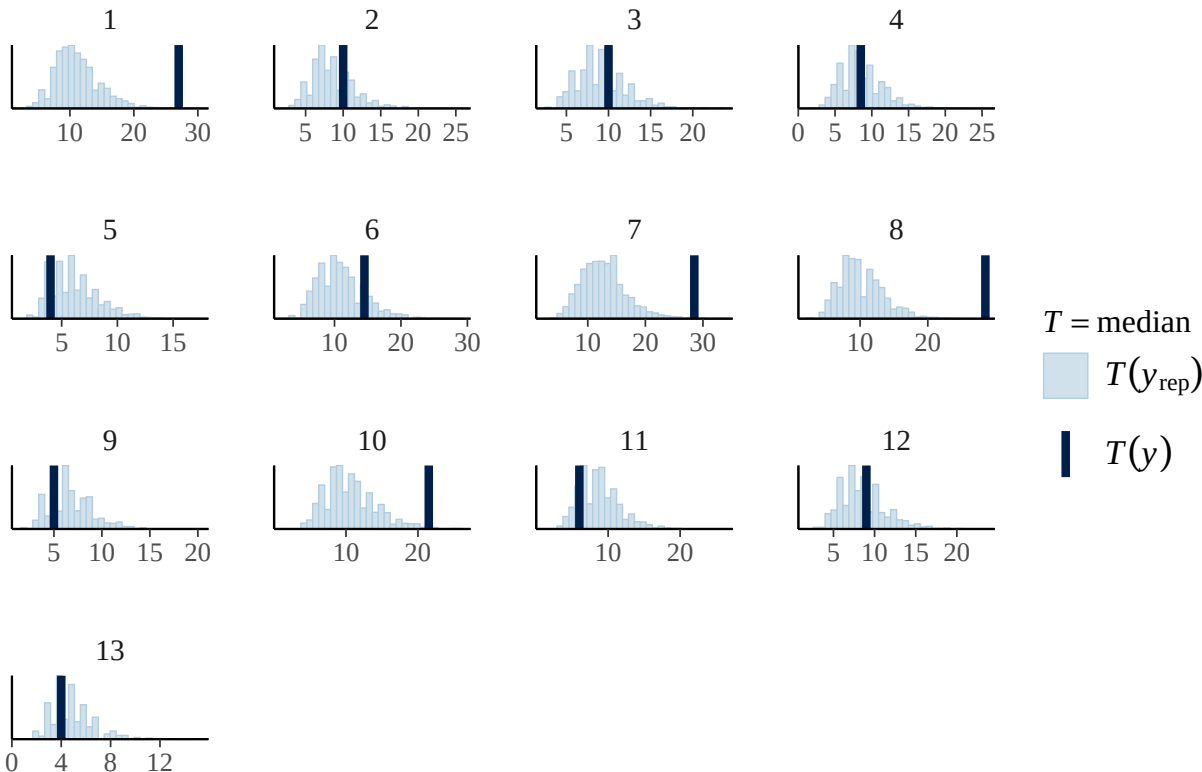
#replicates from model
y_rep <- extract(res_q1_m3)$y_rep # S x N

#group: based on farm
group <- eggs$farm_id

#median
ppc_stat_grouped(y = y_obs, yrep = y_rep, group = group, stat = "median") +
  ggtitle("Posterior predictive check: Median per farm")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```


Posterior predictive check: Median per farm



We perform posterior predictive checks by comparing the observed median of number of eggs per farm with the medians computed from replicated samples obtained from the model.

The light blue histogram shows the distribution of medians from the replicated data, while the dark blue vertical line represents the observed median. A plot has been created for each farm.

We observe that for some farms (2, 3, 4, 5, 6, 9, 11, 12, 13), the observed median fall in the middle of the range of the posterior predictive samples, suggesting quite a good fit of the model in those groups.

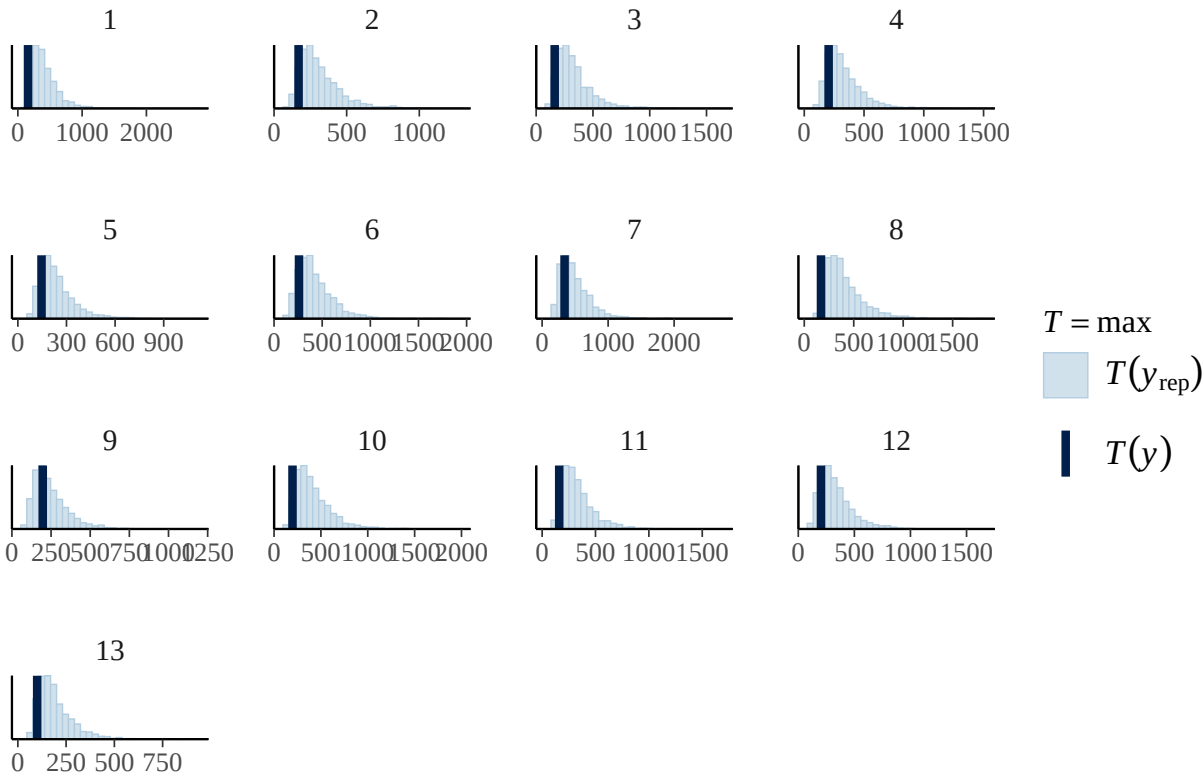
However, for other farms (1, 7, 8, 10), the observed median lies outside the range of the replicated medians. This indicates a potential lack of fit for these specific groups. In particular, all these problematic cases show that the replicated data are underestimating the observed median, indicating that our replicated samples put too much probability on low count of eggs.

These discrepancies may arise from differences across farms not fully captured by the current random effects structure.

```
#max
ppc_stat_grouped(y = y_obs, yrep = y_rep, group = group, stat = "max") +
  ggtitle("Posterior predictive check: Max per farm")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```

Posterior predictive check: Max per farm



Now we compare the observed maximum with the posterior maximums from the samples.

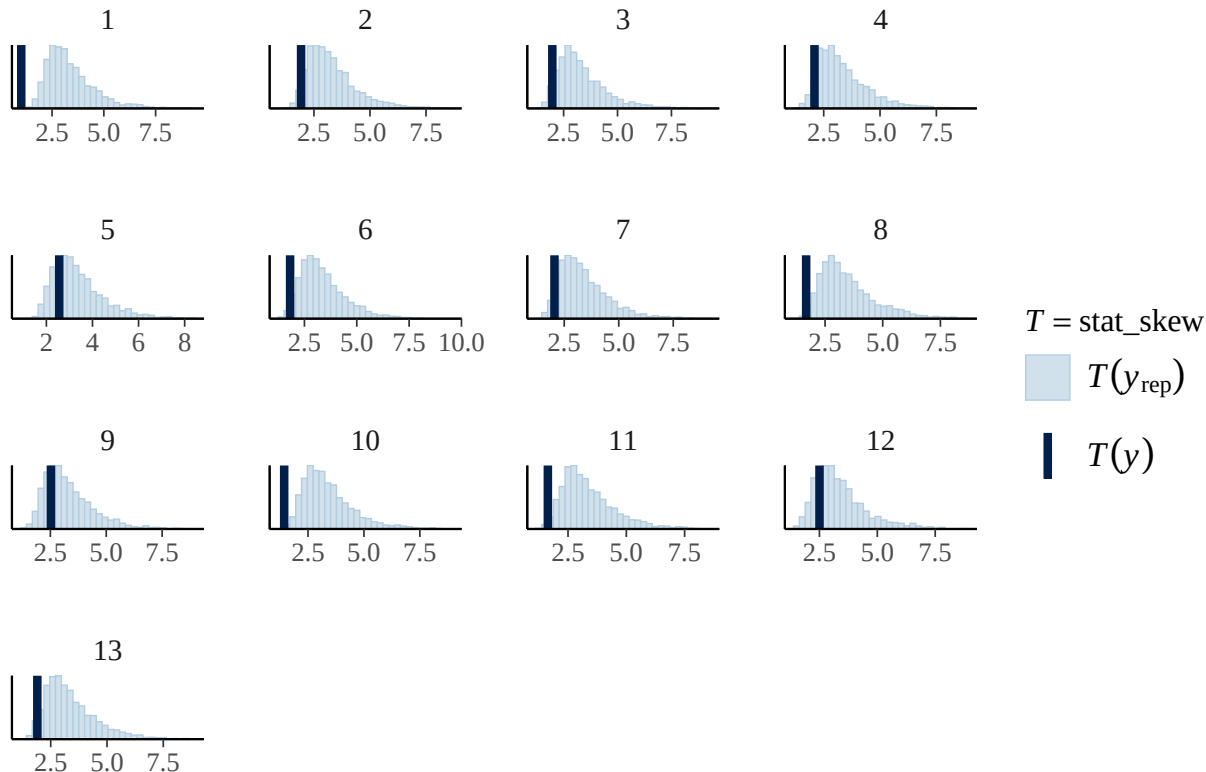
The observed maximum tend to fall within the posterior distributions range, but the observed maximum in all the farms is close to the left bound of the replicated data. Meaning that we are always (largely, looking at the x-axis values) overestimating the maximum.

```
#function for skewness
stat_skew <- function(x) skewness(x)

#skewness
ppc_stat_grouped(y = y_obs, yrep = y_rep, group = group, stat = stat_skew) +
  ggtitle("Posterior predictive check: Skewness per farm")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```

Posterior predictive check: Skewness per farm



This plots evaluate if the asymmetry (skewness) in the observed distribution of egg counts per farm is well captured by the model.

In most farms, the observed skewness is located in the lower tail of the posterior predictive distribution. This indicates that the model tends to overestimate the asymmetry in the data, meaning that it expects more extreme right-skewed outcomes than what is actually observed. This is connected to the overestimation of the maximum seen before.

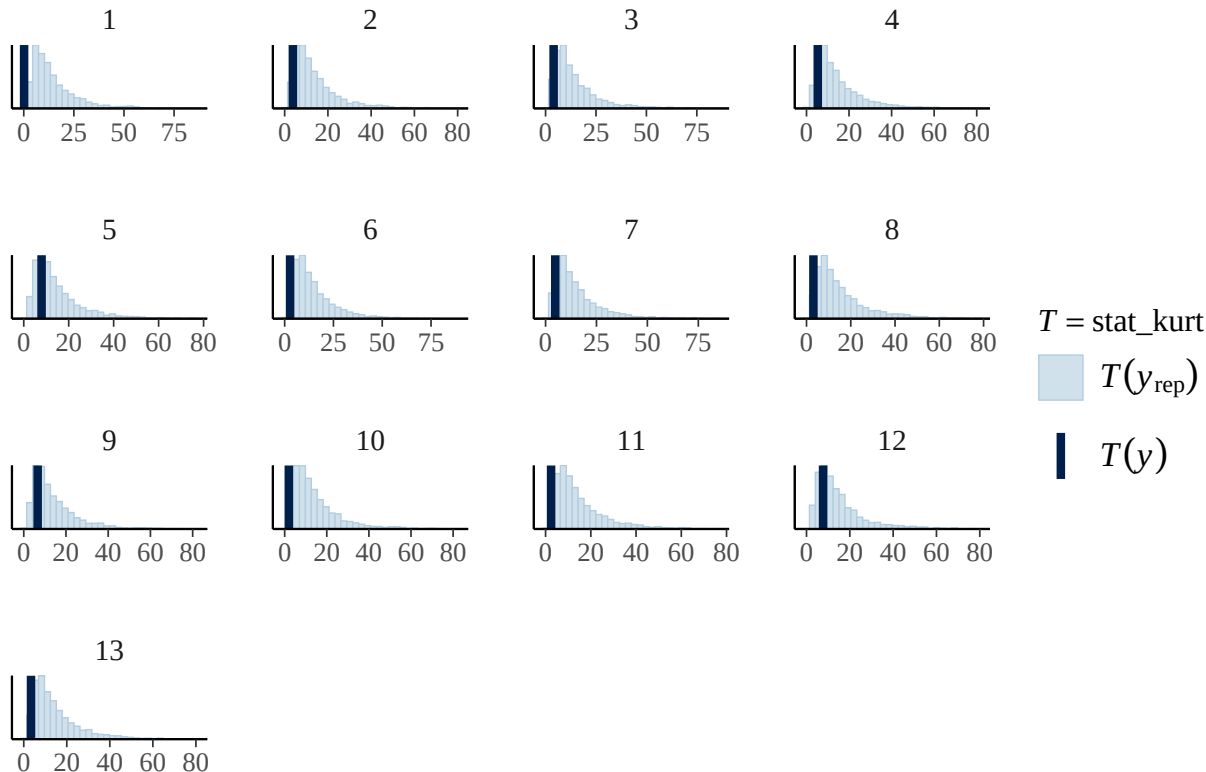
However, since the observed values remain within the range predicted by the model, in each farm, we can conclude that the overall fit is still reasonable.

```
#function for kurtosis
stat_kurt <- function(x) kurtosis(x)

#kurtosis
ppc_stat_grouped(y = y_obs, yrep = y_rep, group = group, stat = stat_kurt) +
  ggtitle("Posterior predictive check: Kurtosis per farm")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```

Posterior predictive check: Kurtosis per farm



This check assesses whether the tails of the distribution of egg counts per farm are consistent with the posterior predictive distributions generated by the model.

The observed kurtosis values generally fall within the range of the simulated values, in most of the cases in the range where the highest frequency of the simulated data is. However, the kurtosis of the simulated samples tends to be higher than that of the observed data. This suggests that the model employs a distribution with heavier tails than those present in the actual data (consistent with what has been observed until now).

```
#min
npred <- nrow(y_rep)
min_per_farm <- matrix(NA, nrow = npred, ncol = J)

#for each farm compute the min in each replication
for (j in 1:J) {
  idx_j <- which(eggs$farm_id == j) # index of position for farm j
  min_per_farm[, j] <- apply(y_rep[, idx_j], 1, min)
}

#names for farms
colnames(min_per_farm) <- paste0("Farm_", 1:J)

cat("proportion of 0 as min in the replicates for each farm \n")
```

```
## proportion of 0 as min in the replicates for each farm
```

```
#for each farm, considering all patch of land, in each
#replication the lowest number of observed eggs is always 0:
apply(min_per_farm, 2, function(x) mean(x == 0))
```

```
## Farm_1 Farm_2 Farm_3 Farm_4 Farm_5 Farm_6 Farm_7 Farm_8 Farm_9 Farm_10
##      1      1      1      1      1      1      1      1      1      1
## Farm_11 Farm_12 Farm_13
##      1      1      1
```

```
#look at observed results now:
min_obs_per_farm <- tapply(eggs$num.eggs, eggs$farm_id, min)
cat("min for each farm in replicated data:")
```

```
## min for each farm in replicated data:
```

```
min_obs_per_farm
```

```
## 1 2 3 4 5 6 7 8 9 10 11 12 13
## 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
#same from data, we are replicating well, no plot bcs it was useless.
```

The model always predicts a minimum of zero for each farm across all replications, which is consistent with the observed data where zero is the minimum in each farm. No plot is shown here, as the result is clear: the minimum is always zero in both observed and simulated datasets.

```
#select part of the samples (100)
sampled_row <- sample(1:nrow(y_rep), 100)
y_rep_sel <- y_rep[sampled_row,]

par(mfrow = c(2, 2))
#density based on farm
for(j in 1:J){
  idx_j <- which(eggs$farm_id == j) #index of each field in farm j
  y_obs_f_j <- y_obs[idx_j] #observed data for farm j
  y_rep_f_j <- y_rep_sel[, idx_j] # replicated data from selected y_pred

  #empty plot
  plot(0, 0, type = "n",
       xlim = range(y_rep_f_j, y_obs_f_j),
       ylim = c(0, 0.06),
       xlab = "Counts", ylab = "Density",
       main = paste("Farm B", j, "- Density per Draw"))

  #density line for each sample
  for(i in 1:nrow(y_rep_f_j)){
    dens_i <- density(y_rep_f_j[i, ])
    lines(dens_i, col = rgb(0, 0, 1, 0.2))
  }

  #density for observed data
```

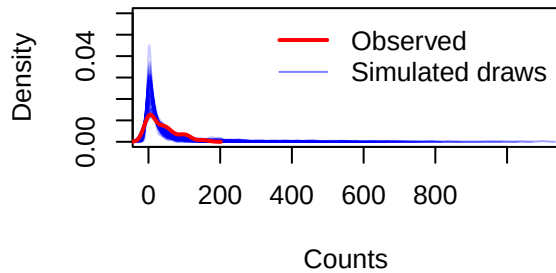
```

lines(density(y_obs_f_j), col = "red", lwd = 2)

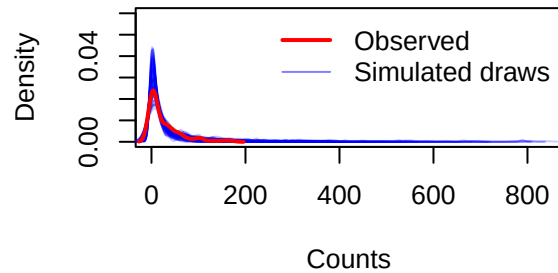
#legend
legend("topright", legend = c("Observed", "Simulated draws"),
      col = c("red", rgb(0,0,1,0.5)), lwd = c(2, 1), lty = c(1, 1), bty = "n")
}

```

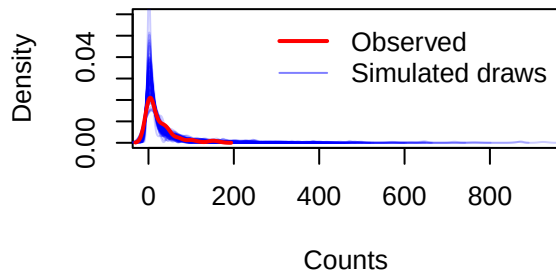
Farm B 1 – Density per Draw



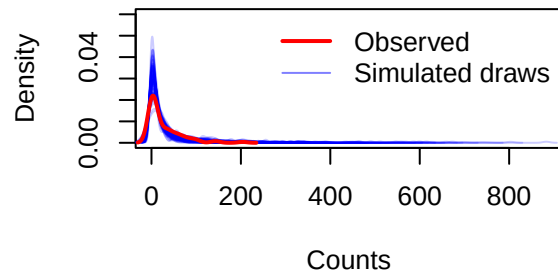
Farm B 2 – Density per Draw



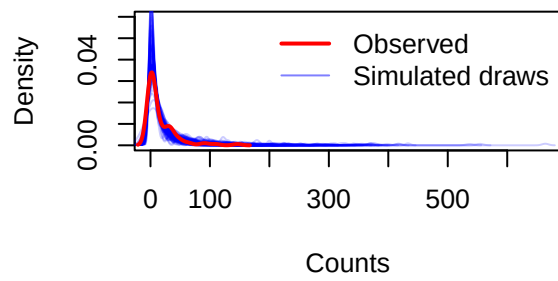
Farm B 3 – Density per Draw



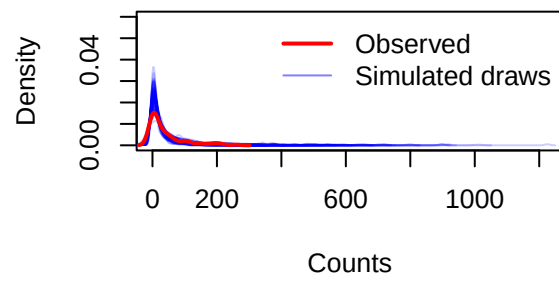
Farm B 4 – Density per Draw



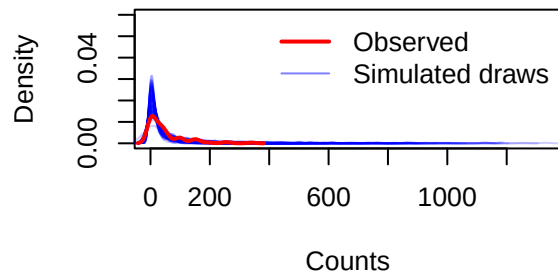
Farm B 5 – Density per Draw



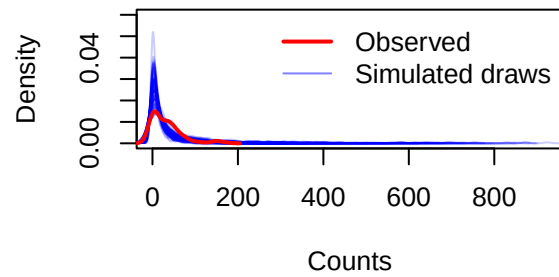
Farm B 6 – Density per Draw



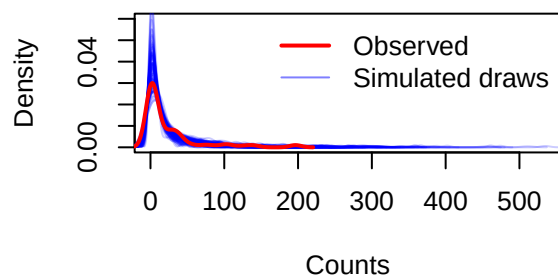
Farm B 7 – Density per Draw



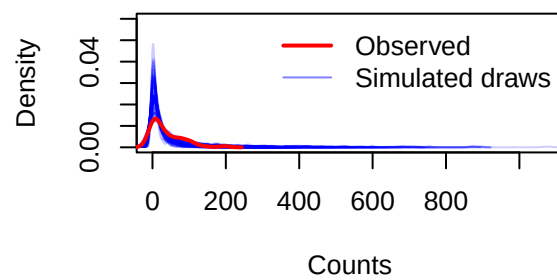
Farm B 8 – Density per Draw



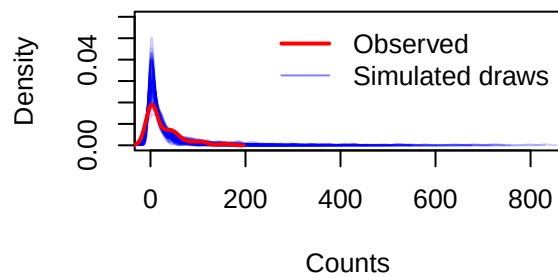
Farm B 9 – Density per Draw



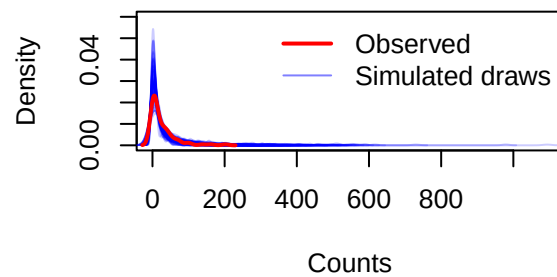
Farm B 10 – Density per Draw



Farm B 11 – Density per Draw

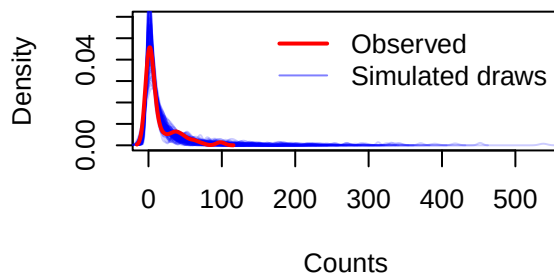


Farm B 12 – Density per Draw



```
par(mfrow = c(1, 1))
```


Farm B 13 – Density per Draw



Although the data are discrete counts, we chose to display density estimates of the distributions (rather than histograms) as a clear way to show the results of the posterior predictive checks.

As shown in the density plots, the model generally captures the overall shape of the distributions well. The observed densities (*red lines*) typically fall within the shape of the simulated densities (*blue lines*), suggesting that the model replicates the main features of the data across farms.

The observed mode (always small values) is usually well predicted, but the model tends to place too much probability mass around it, resulting in sharper peaks than observed.

The previously identified trend of the model to overestimate the maximum is clearly visible in the density plots. This also leads to an overestimation of skewness and kurtosis, and in some cases even the median. This is quite worrying: in the observed data the maximum of eggs observed in each farm is always around 200/300, while in some replication of our data we reach even 1000 and higher.

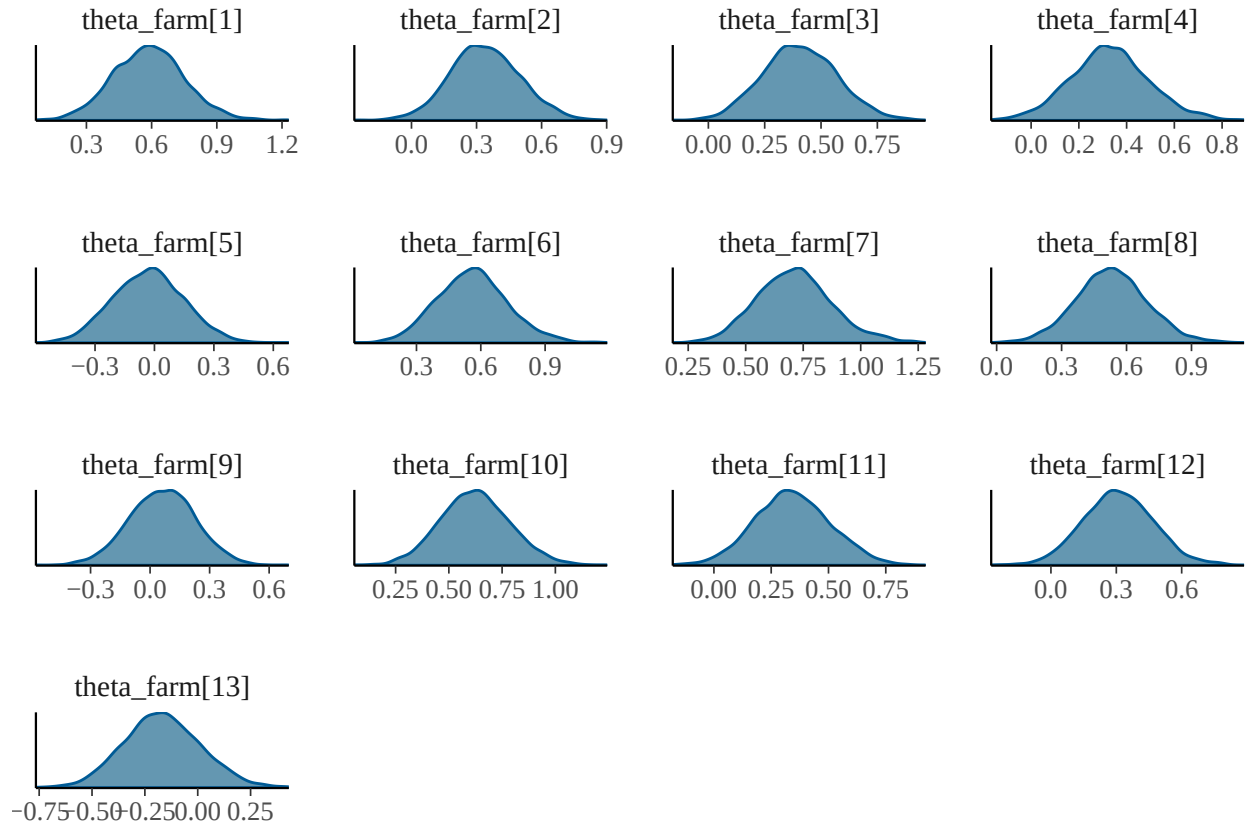
Overall, the check shows good fit in terms of general shape, but also highlights a consistent overestimation of tail heaviness, which could be work for future improvement.

The overestimation of the maximum might happen because of excessive dispersion captured from the model, even if a skeptical prior has been used. Another reason might be related to the fact that we assume the impact of the area just as an offset, meaning we assume that the total number of eggs is always proportional to the area, this might not be always true in reality (in large fields we could be strongly overestimating).

Also, since we are discussing model improvements, it might be interesting to check if the effect of the insecticides is different among the farms, since we are assuming to be the same in each one.

```
#posterior densities random effects
mcmc_dens(res_q1_m3, pars = c("theta_farm[1]", "theta_farm[2]", "theta_farm[3]",
                              "theta_farm[4]", "theta_farm[5]", "theta_farm[6]",
```

```
"theta_farm[7]", "theta_farm[8]", "theta_farm[9]",
"theta_farm[10]", "theta_farm[11]", "theta_farm[12]",
"theta_farm[13]"))
```



The posterior densities for the farm parameters $\theta_{\text{farm}[j]}$ reflect the variability between farm effects on the number of eggs. In particular $\exp(\theta_j)$ is the number of eggs per squared meter that we expect in each farm when no insecticides are used.

These effects seem to be well identified looking at the shape and domain of the posterior density of each parameter.

There is evident variability across farms: while some effects are clearly centered around higher values (farms 1, 7, 10), others are centered closer to zero or even slightly negative (farms 5 and 13). This suggests differences in the farm contribution to the outcome. So in farms like 5 and 13 we expect lower numbers of eggs than in farms like 7 or 10.

The posterior distributions show no signs of multimodality or skewness.

Consider farm B6 and a patch a land with area $30m^2$. Calculate the posterior predictive distribution that the number of caterpillar eggs in that patch of land is equal to zero: i) no insecticide applied, ii) only sprayed insecticide applied, iii) only lead insecticide applied, iv) both insecticides applied.

```
q1_mod4_string <-"
data {
  int<lower=1> N;                // observations
  int<lower=1> K;                // number fixed coeff
  matrix[N, K] X;              // design matrix
  vector[N] log_area;          // offset
```

```

int<lower=0> num_eggs[N];           // response
int<lower=1> J;                     // number of farms
int<lower=1, upper=J> farm_id[N];  // farm for each obs

int<lower=1> N_pred;                // number of different scenarios
matrix[N_pred, K] X_new;          // design matrix for prediction
real log_area_new;                // offset for prediction (log(30))
int farm_pred; // farm id for each prediction (6)

}

parameters {
  vector[K] beta;                  // fixed effects
  real<lower=0> phi;                // overdispersion
  vector[J] theta_farm;            // random effects per farm
  real mu_theta;                  // mean random effects
  real<lower=0> sigma_theta;        // sd random effect
}

model {
  vector[N] eta;                  //linear predictor
  vector[N] mu;                   //mean per observation

  // Priors

  for(i in 1:K){
    beta[i] ~ normal(0, 5);
  }

  phi ~ gamma(2, 0.001);

  //random effects
  mu_theta ~ normal(0, 10);
  sigma_theta ~ gamma(2, 0.1);

  for( j in 1:J){
    theta_farm[j] ~ normal(mu_theta, sigma_theta);
  }

  for(i in 1:N){
    eta[i] = X[i,] * beta + log_area[i] + theta_farm[ farm_id[i] ] ;
    mu[i] = exp(eta[i]);
    num_eggs[i] ~ neg_binomial_2( mu[i], phi);
  }
}

generated quantities {
  vector[N_pred] mu_pred;
  vector[N_pred] p_zero;

  for (i in 1:N_pred) {

```

```

    real eta_pred = dot_product(X_new[i], beta) + log_area_new+theta_farm[farm_pred];
    mu_pred[i] = exp(eta_pred);
    p_zero[i] = exp(neg_binomial_2_lpmf(0 | mu_pred[i], phi)); //compute prob directly in STAN
  }
}
"

# Design matrix for prediction
X_pred <- matrix(
  c(0,0,0, # none
    1,0,0, # sprayed
    0,1,0, # lead
    1,1,1), # both
  ncol = 3, byrow= T)

log_area_pred <- log(30) #predict on 30 squared meters
farm_pred <- 6 #B6 is farm 6 in our farm_id

#data new model
data_q1_m4 <- list(
  N = nrow(eggs),
  K = K0,
  X = X0,
  log_area = log(eggs$area),
  num_eggs = eggs$num.eggs,
  J = J,
  farm_id = eggs$farm_id,
  N_pred = nrow(X_pred), #number of different cases (4)
  X_new = X_pred,
  log_area_new = log_area_pred,
  farm_pred = farm_pred
)

fname_q1_m4 <- 'q1_m4.stan'
cat(q1_mod4_string, file = fname_q1_m4, append = FALSE)

#fit
res_q1_m4 <- stan(
  file = fname_q1_m4,
  data = data_q1_m4,
  chains = 1,
  iter = 8000,
  pars = c("p_zero")) #warm-up as default is half the iter, so we expect 3000

##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.001232 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 12.32 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 8000 [ 0%] (Warmup)
## Chain 1: Iteration:  800 / 8000 [ 10%] (Warmup)

```

```
## Chain 1: Iteration: 1600 / 8000 [ 20%] (Warmup)
## Chain 1: Iteration: 2400 / 8000 [ 30%] (Warmup)
## Chain 1: Iteration: 3200 / 8000 [ 40%] (Warmup)
## Chain 1: Iteration: 4000 / 8000 [ 50%] (Warmup)
## Chain 1: Iteration: 4001 / 8000 [ 50%] (Sampling)
## Chain 1: Iteration: 4800 / 8000 [ 60%] (Sampling)
## Chain 1: Iteration: 5600 / 8000 [ 70%] (Sampling)
## Chain 1: Iteration: 6400 / 8000 [ 80%] (Sampling)
## Chain 1: Iteration: 7200 / 8000 [ 90%] (Sampling)
## Chain 1: Iteration: 8000 / 8000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 39.604 seconds (Warm-up)
## Chain 1: 43.483 seconds (Sampling)
## Chain 1: 83.087 seconds (Total)
## Chain 1:
```

```
pred_samples <- extract(res_q1_m4)$p_zero
```

```
dim(pred_samples) #3000 samples: 1st no insect, 2nd spray, 3rd lead, 4th both
```

```
## [1] 4000 4
```

```
post_prob <- apply(pred_samples, 2, mean)
post_prob
```

```
## [1] 0.1431328 0.2028611 0.1727357 0.2186868
```

We have adapted the previous model to include the computing of the posterior probability directly in the generated quantities block of the Stan model. In particular, we computed the posterior probability of observing zero eggs, $P(y = 0)$, in a patch of 30 m² of land in Farm B6 , under four different scenarios:

1. No insecticide applied: $X_{new_1} = [0 \ 0 \ 0]$
2. Sprayed insecticide only: $X_{new_2} = [1 \ 0 \ 0]$
3. Lead insecticide only: $X_{new_3} = [0 \ 1 \ 0]$
4. Both insecticides applied: $X_{new_4} = [1 \ 1 \ 1]$

The rows represent the design matrix X_{new} for predictions. The values are passed to Stan as data, together with the offset $\log(30)$ (log area of the field we want to do prediction on) and the farm ID ($farm_pred = 6$, that is B6).

Within generated quantities we compute, for each different scenario:

- the linear predictor $\eta_i = X_{new_i} \cdot \beta + \log(30) + \theta_6$,
- the mean $\mu_i = \exp(\eta_i)$,
- the posterior probability $p_zero_i = \exp(\text{neg_binomial_2_lpmf}(0 \mid \mu_i, \phi))$.

We obtain 4000 posterior samples of $P(y = 0)$ for each scenario (8000 iterations, 1 chain, default 50% warmup). To obtain the final estimate we take the mean posterior probability from the samples, for each scenario.

The posterior estimates are:

Insecticide Treatment	Posterior $P(y = 0)$
No insecticide	14.3%
Sprayed only	20.2%
Lead only	17.2%
Both insecticides applied	21.8%

These results show that applying insecticides increases the predicted probability of observing no eggs, with the combined treatment being the most effective at reducing egg presence. This agrees with the previous observations that we have made.